PROFESSIONAL LEARNING GUIDE

Large Language Model Foundations

LLM behavior, tokens, context, prompting, embeddings, vector search, RAG, structured output, tool calling, safety, and production engineering

Prepared for: Saif Ullah
Learning alignment: GenAI foundations, Azure AI application development, RAG, agents, and enterprise solution architecture
Practical focus: Production-ready concepts, .NET-oriented examples, revision material, and hands-on exercises

Detailed notes with diagrams, examples, checklists, summary sheets, interview questions, and revision points.

How to Use This Guide

This guide builds from the internal behavior of an LLM to the architecture of a production enterprise application. Study the concepts in order, run the exercises, then use the final summary and revision section for recall.

> **Recommended learning outcome:** Build a small .NET 8 console or Web API application that sends a controlled prompt, requests schema-constrained JSON, validates the response, and records token usage and latency.

Contents

01	LLM mental model and behavior
02	Tokens and tokenization
03	Context windows and context engineering
04	Prompt design and iterative development
05	Embeddings and semantic similarity
06	Vector search, chunking, and retrieval
07	Retrieval-Augmented Generation
08	Structured output and validation
09	Tool calling and agents
10	Hallucinations, safety, and security
11	Production architecture and evaluation
12	Hands-on labs, summary, and revision

1. LLM Mental Model

What an LLM does

A Large Language Model is a neural network trained to predict the next token based on tokens already present in its input. Generation repeats this prediction step until the model reaches a stopping condition.



At each step, the model produces a probability distribution across its vocabulary. A decoding strategy selects one token, appends it to the sequence, and runs the process again.

Important behavioral implications

- **Probabilistic:** the same prompt can produce different valid responses.
- **Pattern-based:** fluent language does not guarantee factual correctness.
- **Context-dependent:** wording, examples, retrieved documents, and prior messages affect the response.
- **Non-deterministic by default:** sampling settings can increase or reduce variation.
- **No durable memory by default:** an API call only sees data supplied in its active context unless the application adds external storage.

Training versus inference

Stage	Purpose	What happens
Pretraining	Learn broad language patterns	The model learns statistical relationships from large text collections.
Alignment / instruction tuning	Improve instruction following	The model is adjusted using curated examples and preference or safety signals.
Inference	Answer a live request	The trained model predicts tokens from the supplied prompt and active context.

Key caution: A polished answer can still be wrong. Fluency is a language quality signal, not an evidence signal.

Generation controls

Control	Effect	Practical use
Temperature	Changes randomness in token selection	Lower for extraction and classification; higher for ideation.
Top-p	Restricts sampling to a probability mass	Alternative mechanism for controlling diversity.
	Caps generated length	Protects cost and prevents excessive responses.

Maximum output tokens		
Stop sequence	Ends generation on a marker	Useful for controlled text protocols.

2. Tokens and Tokenization

Definition

A token is a model-specific unit of text. It may be a complete word, part of a word, punctuation, whitespace combined with text, a character, or a special control token.

```
Text:       "authentication failed"
Possible:   ["authentication", " failed"]

Text:       "unbelievable"
Possible:   ["un", "believ", "able"]
```

The exact split depends on the tokenizer associated with the selected model. Never assume token counts will remain identical when switching models.

Why tokens matter

- **Cost:** providers commonly measure input and output usage in tokens.
- **Capacity:** context limits are expressed in tokens.
- **Latency:** long inputs require more processing and long outputs require more generation steps.
- **Truncation:** excess content may need to be removed, summarized, or retrieved selectively.
- **Multilingual behavior:** token efficiency varies across languages and scripts.

```
Total request usage = system instructions + conversation history + user input + retrieved content + tool results + generated output
```

Token budget design

Reserve space deliberately rather than filling the entire model context with source material.

Budget component	Example purpose
System and policy instructions	Application behavior, safety, and output contract
User request	Current task and explicit requirements
History or state	Relevant earlier decisions
Retrieved evidence	Selected document chunks
Output reserve	Space required for the complete response
Safety margin	Protection against tokenizer estimation differences

Practical controls

- Estimate token count using the tokenizer recommended for the deployed model.
- Record prompt tokens, completion tokens, total tokens, latency, and model identifier.
- Remove duplicated instructions and irrelevant history.

- Summarize older conversation state into a compact, validated representation.
- Use retrieval instead of injecting an entire document library.

3. Context Windows and Context Engineering

Context window

The context window is the maximum token capacity available to the model for a request-response cycle. The input and generated output must fit within that total capacity.

```
Available output capacity = model context limit - input tokens - reserved overhead
```

What may occupy the context

- System and developer instructions
- User messages and conversation history
- Retrieved documents and citations
- Tool definitions and tool results
- Few-shot examples
- The model's developing answer

Context is not memory

Context is temporary working information made available to the current inference. Durable conversation memory, user preferences, audit history, or workflow state must be managed by the application and selectively reintroduced.

Common long-context problems

Problem	Effect	Mitigation
Irrelevant context	Distracts the model and increases cost	Retrieve only relevant chunks and apply metadata filters.
Conflicting instructions	Produces unstable behavior	Establish instruction hierarchy and remove duplicates.
Lost information	Important details receive weak attention	Place requirements clearly, summarize evidence, and test location sensitivity.
Context overflow	Input fails or content is truncated	Budget tokens, chunk, summarize, or use hierarchical retrieval.
Stale history	Old decisions override current needs	Maintain explicit state and discard obsolete messages.

Context engineering

Context engineering is the broader discipline of selecting, ordering, compressing, and protecting everything placed into the model's active context. Prompt wording is only one part of it.

Production pattern: Keep stable rules in system instructions, current intent in the user message, trusted evidence in clearly delimited context blocks, and strict business logic outside the model.

4. Prompt Design

Core structure

```
ROLE / RESPONSIBILITY
You are assisting with enterprise incident triage.

TASK
Classify the supplied incident and extract required fields.

TRUSTED CONTEXT
Use only the evidence inside <evidence> tags.

CONSTRAINTS
Do not invent missing values. Use null when evidence is absent.

OUTPUT CONTRACT
Return JSON matching the supplied schema.

EVIDENCE
<evidence> ... </evidence>
```

Prompting patterns

Pattern	Description	When useful
Zero-shot	Instruction without examples	Simple or familiar tasks
One-shot	One demonstration	Clarifying style or format
Few-shot	Several representative demonstrations	Classification boundaries and consistent formatting
Role prompting	Defines perspective or responsibility	Controlling domain focus and tone
Decomposition	Breaks a complex task into stages	Analysis, extraction, validation, and generation workflows
Self-check request	Asks the model to verify requirements before finalizing	Reducing omissions, while still requiring external validation

Iterative prompt development

Define success › **Draft** › **Test** › **Error analysis** › **Refine** › **Regression test**

Do not make random wording changes. Categorize each failure: missing data, unsupported claim, wrong tone, incorrect classification, excessive length, invalid schema, or sensitivity to input wording. Modify the relevant prompt component and re-run a stable test set.

Prompt anti-patterns

- Vague instructions such as “make it good.”

- Conflicting constraints such as “be comprehensive” and “use 30 words.”
- Including untrusted source text without delimiters.
- Using the model to enforce rules that code can enforce exactly.
- Optimizing against a single happy-path example.
- Putting secrets, credentials, or unnecessary personal data into prompts.

5. Embeddings

Definition

An embedding is a numerical vector representing semantic characteristics of text or another supported content type. Similar meanings tend to produce vectors located near one another in the embedding space.

```
"ATM cash dispenser fault"  -> [ 0.18, -0.07, 0.42, ... ]
"cash unit failure"         -> [ 0.16, -0.05, 0.39, ... ]
```

Embeddings are not generated answers

An embedding model converts content into vectors. A generative model produces text. Embeddings support retrieval, matching, clustering, recommendation, and anomaly or duplicate detection.

Similarity measurements

```
Cosine similarity = (A · B) / (||A|| × ||B||)
```

Cosine similarity compares vector direction. Other systems may use dot product or Euclidean distance. The index and embedding model must be configured consistently.

Embedding design considerations

- **Model consistency:** index documents and queries with compatible embeddings.
- **Dimensionality:** vector size affects storage and index operations.
- **Normalization:** follow the selected similarity metric and service guidance.
- **Versioning:** changing embedding models may require re-indexing.
- **Privacy:** embeddings can still represent sensitive source content and require access control.
- **Metadata:** retain document ID, source, section, timestamps, classification, and authorization attributes.

Common use cases

Semantic search
Find meaning-related content even when exact terms differ.

RAG retrieval
Select relevant evidence for generation.

Clustering
Group incidents, documents, or feedback by similarity.

Deduplication
Identify near-duplicate tickets or knowledge articles.

6. Vector Search, Chunking, and Retrieval

Why chunk documents

Embedding an entire long document into one vector can hide local meaning. Chunking divides the source into retrievable units while retaining enough context to remain understandable.

Chunking strategies

Strategy	Strength	Risk
Fixed length	Simple and predictable	May split logical sections
Fixed length with overlap	Preserves boundary information	Duplicates retrieved text and increases storage
Structure-aware	Uses headings, paragraphs, or sections	Depends on reliable document parsing
Semantic	Groups meaning-related sentences	More complex and potentially expensive
Hierarchical	Retrieves summaries then detailed children	Requires additional indexing and orchestration

Retrieval methods

- **Keyword search:** strong for exact IDs, error codes, names, and uncommon terms.
- **Vector search:** strong for conceptual similarity and paraphrases.
- **Hybrid search:** combines lexical and vector signals.
- **Metadata filtering:** restricts scope by source, product, date, tenant, security label, or user authorization.
- **Reranking:** applies a stronger relevance model to the initial candidate set.

Retrieval quality metrics

- **Recall@K:** whether relevant evidence appears in the top K results.
- **Precision@K:** how much of the top K is relevant.
- **Mean Reciprocal Rank:** how early the first relevant result appears.
- **Grounded-answer rate:** whether the final answer is supported by retrieved evidence.

> **Operational lesson:** A generation problem may actually be a retrieval problem. Diagnose whether the correct evidence was retrieved before rewriting the prompt.

7. Retrieval-Augmented Generation (RAG)

Purpose

RAG supplies selected external evidence to the model at inference time. It helps the application answer from enterprise documents or current data without retraining the base model for every content change.



Ingestion pipeline

1. Collect approved source documents.
2. Extract text and structural metadata.
3. Clean or normalize content.
4. Split content into meaningful chunks.
5. Create embeddings.
6. Store vectors, searchable text, metadata, and source references.
7. Apply access-control fields and lifecycle policies.

Query pipeline

1. Receive the user request.
2. Apply authorization and scope.
3. Optionally rewrite or expand the search query.
4. Run keyword, vector, or hybrid retrieval.
5. Filter and rerank candidates.
6. Build a bounded evidence context.
7. Generate an answer that cites or points to sources.
8. Validate grounding, format, and policy compliance.

RAG failure modes

Failure	Typical cause	Response
No relevant chunk	Poor parsing, chunking, query, or metadata	Inspect retrieval traces and improve the ingestion or search stage.
Correct chunk ranked low	Weak similarity or lexical mismatch	Use hybrid retrieval, query rewriting, or reranking.
Unsupported answer	Prompt permits open-world generation	Require evidence-only answers and an explicit insufficient-evidence result.
Unauthorized content	Security filtering applied too late	Filter before retrieval results enter model context.
Too much evidence	Large K or oversized chunks	Reduce, deduplicate, compress, or rerank context.

8. Structured Output

Why it matters

Free-form text is difficult to integrate reliably with APIs, databases, workflow engines, and user interfaces. Structured output establishes a machine-readable contract.

```
{
  "incidentId": "INC-10452",
  "category": "Authentication",
  "severity": "High",
  "summary": "User authentication fails after policy update.",
  "evidence": ["Log entry 2026-09-08T07:42:10Z"],
  "requiresHumanReview": true
}
```

JSON Schema example

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["category", "severity", "summary"],
  "properties": {
    "category": { "type": "string" },
    "severity": { "type": "string", "enum": ["Low", "Medium", "High", "Critical"] },
    "summary": { "type": "string", "maxLength": 500 },
    "confidence": { "type": ["number", "null"], "minimum": 0, "maximum": 1 }
  }
}
```

Validation layers

1. **Syntax:** confirm valid JSON.
2. **Schema:** check types, required fields, enums, and additional properties.
3. **Business rules:** validate cross-field logic and permitted values.
4. **Grounding:** verify claims against supplied evidence.
5. **Security:** sanitize content before rendering or executing downstream actions.

.NET contract example

```
public sealed record IncidentClassification(
    string Category,
    string Severity,
    string Summary,
    double? Confidence,
    bool RequiresHumanReview);

// Deserialize only after schema validation.
var result = JsonSerializer.Deserialize<IncidentClassification>(
    json,
    new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
```

Recommended approach: Use provider-supported structured-output or schema features when available, followed by your own schema and business-rule validation. Never trust a successful model response as equivalent to a valid business transaction.

9. Tool Calling and Agents

Tool calling

Tool calling allows the model to select a predefined operation and produce structured arguments. The application remains responsible for validating the call, obtaining approval when required, executing the operation, and returning the result.



Agent components

- Model for reasoning and language generation
- Instructions and policies
- Tools or functions
- State and memory managed by the application
- Planning or orchestration loop
- Approval boundaries
- Observability and audit logs

Safe execution rules

- Use allow-listed tools and strict parameter schemas.
- Validate authorization independently from the model.
- Require confirmation for destructive, financial, external-communication, or high-impact actions.
- Apply idempotency keys where repeated calls could duplicate changes.
- Set timeouts, retry limits, and maximum tool-call depth.
- Log requested action, validated arguments, authorization decision, execution result, and correlation ID.

10. Hallucinations, Safety, and Security

Hallucination

A hallucination is generated content that is unsupported, incorrect, or fabricated while appearing plausible. It may arise from missing evidence, ambiguous instructions, weak retrieval, conflicting context, or the probabilistic nature of generation.

Mitigation stack

- Ground responses in trusted, authorized evidence.
- Request citations or evidence identifiers.
- Allow the model to return “insufficient evidence.”
- Use structured outputs with nullable fields rather than invented values.
- Validate critical claims and calculations outside the model.
- Place a human reviewer in high-risk workflows.

Prompt injection

Prompt injection occurs when untrusted content attempts to alter application instructions, expose data, or trigger unintended actions. Documents, web content, emails, tickets, and tool results must be treated as data, not trusted instructions.

> **Security boundary:** The model must not decide authorization. Retrieval permissions, data access, action approval, secrets management, and audit controls belong to deterministic application and platform layers.

11. Production Architecture

Client › API gateway › Authorization › Orchestrator › Retrieval / tools › LLM › Validation

Required production controls

Area	Controls
Reliability	Timeouts, bounded retries, circuit breakers, fallback behavior, idempotency
Cost	Token budgets, model routing, caching, context reduction, usage quotas
Performance	Streaming, parallel retrieval, response limits, cache strategy
Security	Managed identity, secret vault, RBAC, network controls, content filtering
Privacy	Data minimization, retention policy, redaction, tenant isolation
Observability	Correlation IDs, token usage, latency, retrieval traces, tool calls, validation failures
Quality	Evaluation dataset, regression tests, groundedness checks, human review

Evaluation framework

- **Task success:** Did the application achieve the required business outcome?
- **Correctness:** Are extracted or generated facts accurate?
- **Groundedness:** Is each factual claim supported by provided evidence?
- **Completeness:** Are mandatory fields and requested topics present?
- **Format validity:** Does output pass schema and business rules?
- **Safety:** Did the application resist injection and avoid restricted disclosure?
- **Operational quality:** Are latency, cost, and failure rate acceptable?

Build an evaluation dataset containing normal cases, edge cases, adversarial inputs, missing evidence, conflicting data, long documents, multilingual content, and expected abstentions. Run it whenever the prompt, model, retrieval configuration, or application logic changes.

12. Hands-on Learning Labs

Lab 1 - Observe LLM behavior

- Send the same classification prompt several times.
- Compare response consistency under different generation settings.
- Record outputs and identify variation that would break automation.

Lab 2 - Token budgeting

- Measure system, user, retrieved, and output tokens separately.
- Add duplicate context and observe cost and latency changes.
- Implement an input budget and output reserve.

Lab 3 - Prompt iteration

- Create a first prompt for summarizing an incident log.
- Evaluate missing fields, unsupported conclusions, and length.
- Improve one issue per prompt version and record results.

Lab 4 - Embedding search

- Create a small sanitized knowledge set containing incident summaries and resolutions.
- Generate embeddings and store vectors with metadata.
- Compare keyword, vector, and hybrid retrieval on paraphrased queries.

Lab 5 - RAG with citations

- Retrieve the top candidate chunks.
- Require the answer to reference source IDs.
- Return insufficient evidence when retrieval has no valid answer.

Lab 6 - Schema-validated .NET output

- Define a C# record and JSON Schema.
- Request structured incident classification.
- Validate syntax, schema, enums, business rules, and evidence fields.
- Log invalid responses and use a bounded repair or retry path.

Lab 7 - Security tests

- Insert malicious instructions inside a sample document.
- Confirm the application treats them as untrusted content.
- Test unauthorized retrieval, unsafe tool arguments, and approval enforcement.

Master Summary

Concept	Meaning	Enterprise importance
LLM	Probabilistic next-token generator	Provides flexible language and reasoning capabilities
Token	Model-specific unit of text	Determines cost, capacity, and latency
Context window	Total active token capacity	Constrains instructions, evidence, history, and output
Prompt	Instructions and task context	Guides behavior but does not replace validation
Embedding	Vector representation of semantic features	Enables semantic retrieval and matching
Vector search	Retrieval by vector similarity	Finds related meaning beyond exact keywords
RAG	Retrieve evidence before generation	Grounds answers in enterprise information
Structured output	Schema-constrained machine-readable response	Supports reliable application integration
Tool calling	Model proposes structured function use	Connects language intent to controlled operations
Evaluation	Repeatable measurement against test cases	Prevents regressions and supports production readiness

Architecture in one line: Authorize the user, retrieve trusted evidence, construct a bounded prompt, generate a schema-constrained response, validate it deterministically, and record the complete operational trace.

Rapid Revision Points

- LLMs generate token sequences from probabilities; fluency does not prove truth.
- Input and output share the context budget.
- Context engineering determines what the model sees; application memory is external.
- Prompt quality improves through testing and error analysis.
- Embeddings encode semantic characteristics for similarity-based retrieval.
- Hybrid retrieval is useful when both concepts and exact identifiers matter.
- RAG quality depends on ingestion, chunking, retrieval, reranking, and generation.
- Structured output still requires schema and business-rule validation.
- The model can propose tools, but code enforces authorization and approval.
- Production quality requires evaluation, observability, security, privacy, and cost controls.

Revision Questions

1. Why can a fluent LLM response still be factually incorrect?
2. Which parts of an LLM request consume the context window?
3. Why should exact word limits be enforced in code?
4. What is the difference between context and durable memory?
5. When should few-shot prompting be preferred over zero-shot prompting?
6. What does an embedding represent?
7. Why might hybrid search outperform vector-only search for incident data?
8. How does chunk size affect retrieval quality?
9. What is the difference between a retrieval failure and a generation failure?
10. Which validation layers should follow structured output?
11. Why must authorization remain outside the model?
12. Which metrics would you track for a production RAG service?

Interview and Architecture Prompts

- Design a secure RAG solution for banking support runbooks.
- Explain how you would prevent unauthorized chunks from entering model context.
- Describe a .NET validation and retry strategy for schema-constrained output.
- Compare long-context prompting with selective retrieval.
- Explain how you would detect whether a quality regression came from the prompt, model, or index.

This is an original educational guide prepared from the concepts discussed in the conversation. Model limits, pricing, SDK syntax, and service features vary by provider and deployment; verify those implementation-specific details against the documentation for the model and platform you deploy.